PE Document Intelligence Platform

Revised System Design — Unified LangGraph Pipeline

Single + Bulk on one graph • Confidence-gated HITL review • Shared middleware stack (PII, retry, rate-limit)

Prepared for VP Engineering | April 2026

Agenda

What's new in the revised design, and what stayed the same

01 **What Changed & Why**
Design deltas since v1

02 **Platform Overview**
Capabilities and modes

03 **High-Level Architecture**
End-to-end block diagram

04 **Unified LangGraph Pipeline**
Single + bulk on one graph

05 **Confidence Gates & HITL**
Review routing and interrupts

06 **Pipeline Nodes**
Parse → ingest, node-by-node

07 **Common Middleware Stack**
PII, retry, rate-limit, run-guard

08 **Classification & Extraction**
Schemas, judging, review flags

09 **Summarization & RAG**
PE attributes, hybrid search

10 **Data Agent & Analytics**
NL-to-SQL with schema guards

11 **Audit & Event Bus**
Non-blocking + SSE progress

12 **Storage & Frontend**
PostgreSQL, Weaviate, React 19

What Changed & Why

Key architectural deltas since the previous design deck

BEFORE (v1)

Two orchestrators
DeepAgent (single-doc) + separate LangGraph (bulk)

Middleware per agent
Each subagent wired its own PII / retry / limits

Linear flow, no HITL branching
Review step existed only in single-doc UI path

Resume/retry logic split
Different code paths for single vs bulk resume

AFTER (current)

One LangGraph StateGraph
Single pipeline handles single + bulk documents

Common middleware
Centralized PII filter, retry, rate-limit, run-guard

Confidence-based gates
route_after_parse / route_after_extract → interrupt()

Unified PipelineRunner
start / resume / retry_node on one checkpointer

Platform Overview

AI-powered processing for LPA, Subscription Agreements, and Side Letters

Upload & Parse

Reducto Cloud API. PDF/DOCX/XLSX/images. SHA-256 dedup on upload.

Classify

Hybrid filename + content/summary LLM classifier. Confidence 0–100.

Extract

Dynamic Pydantic schema per category. Value + verbatim source quote.

Judge

Separate LLM pass scoring each field high/medium/low.

Summarize

17 PE attributes preserved. No truncation.

RAG Retrieval

Hybrid BM25 + vector, cross-encoder re-rank, cited answers.

Analytics

NL-to-SQL data agent with auto chart recommendation.

Audit

Non-blocking fire-and-forget + SSE progress stream.

Two Processing Modes — Same Graph

Single Document: user-driven, editable at parse + extraction review gates. **Bulk Automated:** concurrent processing (asyncio.Semaphore, default 10), auto-ingest on high confidence.

High-Level Architecture

React 19 frontend → FastAPI → unified LangGraph pipeline → external services

React 19 + Vite + TypeScript | TanStack Query | SSE (useEventStream) | TipTap editor

FastAPI (Python 3.13) | REST • SSE • CORS • RunGuard (no duplicate doc runs)

Unified LangGraph Pipeline (PipelineRunner + MemorySaver checkpointer)

parse → summarize → classify → extract → ingest → finalize

await_parse_review

await_extraction_review

HITL gates — LangGraph interrupt() when confidence below threshold

Reducto

Document parsing

PostgreSQL 16

Metadata + state

Weaviate 1.30

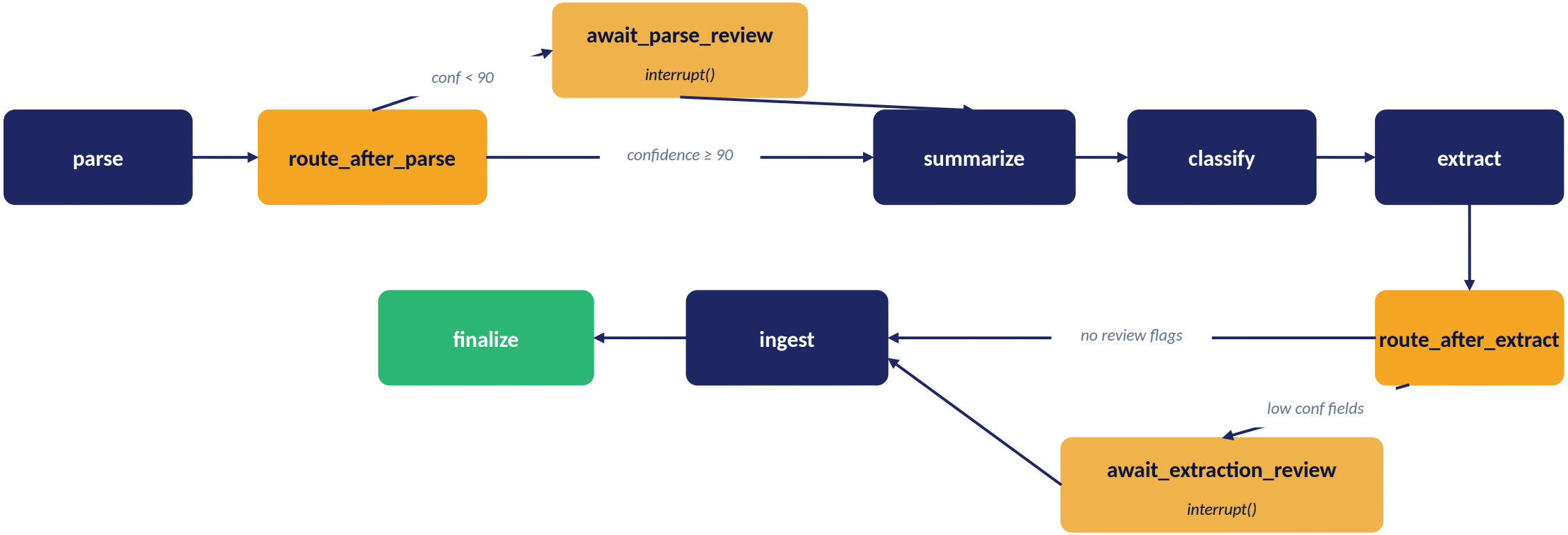
Vector store (hybrid)

OpenAI

GPT-5.4-mini | GPT-5.3-codex

Unified LangGraph Pipeline

Single graph, single runner — single-doc and bulk share every node and every gate



Design Properties

One graph bulk/pipeline.py → StateGraph(DocumentState)

Thread = doc_id PipelineRunner uses document UUID as checkpoint thread

Resumable MemorySaver checkpointer + interrupt_before waits

Concurrent bulk asyncio.Semaphore, DEFAULT_CONCURRENCY = 10

Node reuse Same service classes used by API and graph nodes

Data-driven HITL route_* functions inspect state only — pure logic

Confidence Gates & Human-in-the-Loop

Routing is pure state inspection — gates decide whether to pause or flow through

route_after_parse

Threshold: parse_confidence_pct ≥ 90% (settings.parse_confidence_threshold)

≥ threshold → summarize (automatic)
< threshold → await_parse_review (pause)

User can edit parsed markdown in TipTap, then resume via `PipelineRunner.resume(doc_id)`

route_after_extract

Trigger: any(result.requires_review) set by Judge subagent

no review flags → ingest (automatic)
any flag set → await_extraction_review (pause)

Triggers: LOW confidence, MEDIUM + empty value, required-but-empty

Interrupt Lifecycle

1. **Gate fires** — State-only decision; no LLM call
2. **Status persisted** — documents.status = awaiting_*_review
3. **interrupt()** — LangGraph suspends on thread_id
4. **User edits** — Frontend TipTap or field review form
5. **Runner.resume** — Graph reloads state, re-enters node
6. **Flow resumes** — Continues to next node with fresh data

Pipeline Nodes

Every node is a thin wrapper over a shared service — same code runs single or bulk

Node	Service / Agent	State it writes
parse	ParseService + ReductoClient	parse_confidence_pct, parsed_path, parsed_content
await_parse_review	interrupt() + DB status	status = awaiting_parse_review
summarize	SummarizeService (17 PE attrs)	summary_text
classify	ClassifierAgent (hybrid signals)	classification_result, category_id, category_name
extract	ExtractorAgent + Judge subagent	extraction_results[], judge_results[], requires_review
await_extraction_review	interrupt() + DB status	status = awaiting_extraction_review
ingest	IngestionService → Weaviate	chunks_created
finalize	Status + timings + audit emit	end_time_ms, node_timings

Every node records timing into state.node_timings and emits an event to PipelineEventBus — consumed by SSE stream and audit_logs.

Common Middleware Stack

Cross-cutting concerns centralized — every agent and node call flows through the same layers

Incoming call → RunGuard | PII Filter | Rate Limit | Retry (3x) | PII Log Filter

RunGuard

Per-document asyncio lock. Rejects concurrent runs on the same doc_id. Active-set bookkeeping + non-blocking try-acquire.

PII Filter

Regex + passthrough list. Redacts SSN, phone, email, bank accounts, US addresses. Preserves PE terms (mgmt fee, carry, NAV...). REDACT / MASK / BLOCK strategies.

Rate Limit

AgentRateLimiter tracks per-run LLM and tool calls. Hard caps: 50 LLM / 200 tool. Raises AgentRateLimitError on overflow.

Retry

LLMRetryMiddleware. Retry on 429 / 5xx with exponential backoff (base 1s, max 60s) + optional fallback model. Non-retryable 4xx surface immediately.

PII Log Filter

Logging handler wraps every emitted record through the PII filter — audit_logs and log files never store raw PII.

Observability

OpenTelemetry spans around every node + middleware. LangSmith tracing enabled on pipeline runs. Structured JSON logs.

Applied uniformly: every DeepAgent subagent call and every LangGraph node call passes through the same middleware stack — no per-agent wiring, no drift.

Classification, Extraction & Judging

Hybrid signals for classification; dynamic schema + separate LLM judge for extraction

Classifier

Signal 1 — Filename heuristic

`LPA_*` → Limited Partnership Agreement
`Sub_*` → Subscription Agreement
`Side_Letter_*` → Side Letter

Signal 2 — Content/summary LLM

Prefers summary if available; falls back to full parsed markdown (no truncation).
Few-shot examples per category. Structured output: ClassificationResult.
Confidence 0–100 → drives route_after_parse decision downstream.

90–100	70–89	50–69	<50

Confidence buckets → supports vs. ambiguous vs. guess

Extractor + Judge

Dynamic Pydantic schema per category

Each field produces TWO values: `value` + `value_source` (verbatim quote).
Schema loaded from extraction_schemas + extraction_fields tables.

Judge — separate LLM call

Scores every field **high / medium / low** on source match, format validity, completeness.
requires_review = LOW OR (MEDIUM + empty) OR (required + empty).
Any flag → route_after_extract pauses at await_extraction_review.

LPA seeded fields (8):

Fund Name	General Partner	Mgmt Fee	Carried Interest
Pref Return	Fund Term	Commit Period	Governing Law

Summarization & RAG Retrieval

PE-attribute-preserving summaries feed retrieval; answers always cite source chunks

Summarizer

Full parsed content. No truncation. 17 PE attributes enforced by system prompt.

Fund Name	General Partner	Limited Partners
Mgmt Fee	Carried Interest	Preferred Return
Fund Term	Commit Period	Distribution Waterfall
Governing Law	Capital Commitment	Key Person
Clawback	Investor Reps	Fee Discounts
MFN Clauses	Co-invest	

RAG Pipeline

1. **Chunk**
   LangChain SemanticChunker, 512 tok / 100 overlap, markdown-header-aware
2. **Embed**
   Weaviate DocumentChunks, text2vec-openai vectorizer
3. **Search**
   Hybrid BM25 + vector, alpha configurable (0=keyword, 1=semantic, 0.5=balanced)
4. **Re-rank**
   Cross-encoder (ms-marco). Top-K default 5
5. **Answer**
   RAG agent composes citation-backed response. Scope: doc / category / global

Data Agent & Analytics

Natural language to SQL, guarded by schema introspection and PII middleware

NL Question
AnalyticsPage + ChatInput

→

Schema Introspect
Live DB schema → LLM sys prompt

→

GPT-5.3-codex
Generates parameterized SQL

→

Execute
SELECT-only whitelist, params binding

→

Chart Rec
Bar / line / pie / table

SELECT-only whitelist

Executor rejects any non-SELECT statement. No DDL, no DML.

Parameterized execution

Values bound as parameters to prevent injection.

PII Filter middleware

Applied to sys prompt + results before logging.

Rate + retry

Same Retry / RateLimit middleware as document agents.

Auto chart recommendation

Agent infers bar/line/pie/table from result shape.

Frontend

ChartRenderer (Recharts). Interactive tables + charts.

Audit Framework & Event Bus

Never block the hot path — audit writes are fire-and-forget; progress streams in real time

fire & forget — calling code never awaits persistence

Non-blocking

Background asyncio loop drains queue; API latency unaffected.

Immutable events

AuditEvent is frozen dataclass — tamper-proof trail.

Real-time SSE

useEventStream hook consumes audit + pipeline progress.

Graceful shutdown

Queue flushed before background thread terminates.

Event types

document.*, parse.*, classification.*, extraction.*, rag.query, analytics.query, bulk.*

Per-job bus

PipelineEventBus subscribes by job_id; isolates bulk streams.

Storage & Frontend

PostgreSQL 16 + Weaviate 1.30 + disk caches. React 19 consumer.

PostgreSQL 16

12 tables, UUID PKs, asyncpg driver, selectinload to avoid N+1

Core

`documents, document_categories, document_summaries`

Extract

`extraction_schemas, extraction_fields, extracted_values`

Bulk

`bulk_jobs, bulk_job_documents`

Memory

`conversation_summaries, memory_entries`

Audit

`audit_logs`

Weaviate + Disk Caches

Weaviate 1.30

```
Collection: DocumentChunks
Vectorizer: text2vec-openai
Search: hybrid (BM25, vector)
```

Disk layout

`data/upload/` originals
`data/parsed/` parsed markdown
`data/extraction/` extraction cache
`data/summary/` summary cache

React 19 Frontend

Vite + TS + Tailwind + TanStack Query

Dashboard	Card grid + detail
Upload	Drag-drop, single + bulk
Parse/Edit	TipTap markdown
Bulk Jobs	Live per-doc progress
RAG Chat	Citation cards
Analytics	Recharts visualization

Design Summary

The revised platform collapses two orchestrators into one — with shared middleware and data-driven HITL gates.

1. **Single LangGraph StateGraph**

   Same graph, same nodes, same checkpointer serves single-doc and bulk flows.

2. **Confidence-gated HITL**

   route_after_parse / route_after_extract pause via LangGraph interrupt() — pure state-driven.

3. **Common middleware**

   PII filter, retry, rate-limit, run-guard, log scrubber applied uniformly to every call.

4. **Unified PipelineRunner**

   start / resume / retry_node — one API for both modes, backed by MemorySaver.

5. **Streaming observability**

   Non-blocking audit + PipelineEventBus → SSE to frontend in real time.